

Simplest approach As defined there are 3 LLM agents (1. Example Generator, 2. Relation Inference Agent, and 3. Evaluator), and we are supposed to update the prompt for Agent 1 and 2, using the feedbacks of Agent 3. For simplicity, now, we will update only the prompt of Agent 2 (relation inference agent), and keep other prompts frozen.

A simple approach is given below that is similar to the work of GEPA:

Given, Training set (Feedback set) $D_{feedback}$, Validation set $D_{validation}$, Test set D_{test} , Initial system S_0 that has prompts $P = \{P_{ex_genertaion}, \dots, P_{inference}\}$, and a initial population $Q = \{S_0\}$.

We run the following steps in each iteration.

1. Sample a System S_x from Q based on some priority.
2. Sample f instances from $D_{feedback}$ (typically, $f = 3$) and use Prompt $P_{inference}$ of S_x for inference on these instances.
3. Do evaluation using $P_{evaluation}$ prompt on the performance on f instances.
4. Mutate i.e. update the prompt $P_{inference}$ based on evaluation; we get new prompt $P'_{inference}$ that forms the system S'_x
5. Run system S'_x on validation set.
6. Add the system S'_x in the population Q . If the size of current population set is over the $MAX_POPULATION$, we remove the weakest system (the one with lowest score on validation set) from the population.

At the end we get our best system from the population Q , that has the highest score on validation set. Using the best system, we run it on the Test set.

The sample prompts are given below:

Evaluation Prompt

You are an good evaluator for a relation extraction inference task.

In this task, a relation defines the connection between two selected entities in a sentence: a subject and an object. The relation describes the role or relationship that the object has with respect to the subject. The task is to infer a yes or no answer based on whether the query sentence expresses this relation between the subject and the object.

You are given below an input instance for relation inference that contains:

- A relation and its description
- A support instance (an example where the relation holds)
- A query sentence
- A ground-truth label (yes/no)
- A yes/no inference made by another LLM

Your task is to explain the likely reasoning that led the other LLM to predict yes or no, using only the given relation description, support instance, and query.

Do not re-evaluate the relation yourself. Instead, explain the LLM's decision in light of the ground-truth label.

- If the prediction matches the label, explain what cues or evidence likely led to that choice.
- If it does not match, explain what misunderstanding, missing evidence, or heuristic likely caused the prediction.

Instance:

'''

Relation: #RELATION#

Relation description: #RELATION_DESCRIPTION#

Support Instance: #SUPPORT_INSTANCE#

Query: #QUERY#

Label: #LABEL#

LLM inference: #INFERENCE#

'''

Output your feedback only inside <f> and </f> tags.

Mutation Prompt

You are an good prompt generator for a relation extraction inference task.

In this task, a relation defines the connection between two selected entities in a sentence: a subject and an object.
The relation describes the role or relationship that the object has with respect to the subject.
The task is to infer a yes or no answer based on whether the query sentence expresses this relation between the subject and the object.

You are given below an prompt that is used by the another LLM to make the inference of the task:
'''
#INFERENCE_PROMPT#
'''

Using this prompt, the other LLM was tested on 3 instances of this task. Below you are given for each of the tasks: the inputs, the inference made by the other LLM, and a feedback.
'''

Task 1
Relation: #RELATION_1#
Relation Description: #RELATION_DESCRIPTION_1#
Support Instance: #SUPPORT_INSTANCE_1#
Query sentence: #QUERY_1#
Ground-truth label: #LABEL_1#
LLM inference: #QUERY_1#
Feedback: #FEEDBACK_1#

Task 2
Relation: #RELATION_2#
Relation Description: #RELATION_DESCRIPTION_2#
Support Instance: #SUPPORT_INSTANCE_2#
Query sentence: #QUERY_2#
Ground-truth label: #LABEL_2#
LLM inference: #QUERY_2#
Feedback: #FEEDBACK_2#

Task 3
Relation: #RELATION_3#

Relation Description: #RELATION_DESCRIPTION_3#
Support Instance: #SUPPORT_INSTANCE_3#
Query sentence: #QUERY_3#
Ground-truth label: #LABEL_3#
LLM inference: #QUERY_3#
Feedback: #FEEDBACK_3#
'''

Read the inputs, outputs, and the feedbacks carefully. Idenfity the problems with the current given prompt.
Your task is to generate a revised form of the prompt so that the other LLM can improve the generalization of the task while using it.
You may include modify, add or remove any instructions provided in the current prompt.
You can put additional details or do anything in the prompt that may help to improve the prediction with generalization.

Output the revised prompt only under <p> and </p> tags.

The original prompt in GEPA paper used for mutation

I provided an assistant with the following instructions to perform a task for me:

'''

<current instruction>

'''

The following are examples of different task inputs provided to the assistant along with the assistant's response for each of them, and some feedback on how the assistant's response could be better:

'''

<Inputs, Outputs and Feedback for minibatch of examples>

'''

Your task is to write a new instruction for the assistant.

Read the inputs carefully and identify the input format and infer detailed task description about the task I wish to solve with the assistant.

Read all the assistant responses and the corresponding feedback. Identify all niche and domain specific factual information about the task and include it in the instruction, as a lot of it may not be available to the assistant in the future. The assistant may have utilized a generalizable strategy to solve the task, if so, include that in the instruction as well.

Provide the new instructions within ''' blocks.

Method	Single / Multi Agent(LLM)	What Is Optimized	Prompt Exploration Strategy	Information Provided in the Meta-Prompt used to generate the new prompt
OPRO [5]	Single	One prompt	Iterative refinement	Parent prompts together with their scalar performance scores
EVOPROMPT [2]	Single	One prompt	Evolutionary search	Parent prompts only ; no scores or task feedback included
PDO [4]	Single	One prompt	Preference-based search	Parent prompts with pairwise win/loss preference outcomes, plus fixed prompt mutation templates
MIPRO [3]	Single / Multi	Multiple prompts	Bayesian search	No parent prompts ; prompts are generated independently from task descriptions or templates
GEPA [1]	Multi	Multiple prompts	Evolutionary search	Current system prompts plus execution and evaluation traces from full system runs

Table 1: Comparison of prompt optimization methods across agent settings, optimization targets, and prompt exploration strategies. More details for each methods are given in Section 2

1 Multi-agent system for Few-shot relation extraction with Optimized prompts

Our goal is the same as our previous work. We aim to improve few-shot relation extraction by adding more examples on top of a 1-shot example.

We design a multi-agent system with three LLM agents. All agents use the same underlying LLM.

The first agent is the **Example Generator**. This agent generates additional examples for a target relation.

The second agent is the **Relation Inference Agent**. This agent performs relation inference as a binary classification task.

The third agent is the **Evaluator (Critic)**. This agent evaluates the outputs of the Example Generator and the Relation Inference Agent.

The Evaluator analyzes performance, inputs, and outputs of the other agents. It also reasons about success and failure cases.

The feedback from the Evaluator is used to update the prompts of the first and second agents. This process is repeated iteratively to improve prompt quality.

2 Summary of different Prompt Optimizer approaches

OPRO: (Simplest approach) OPRO is a single-agent prompt optimization method that iteratively improves a single prompt. It conditions prompt generation on a ranked history of previously evaluated prompts. Each new prompt is generated by reasoning over parent prompts and their scalar performance scores. The optimization proceeds through simple score-guided refinement without evolutionary operators.

EVOPROMPT: It is a single-agent evolutionary prompt optimization framework. It maintains a population of prompts and evolves them using selection and mutation operators. New prompts are generated solely from parent prompts, without access to scores or task feedback during generation. Evaluation results are used only for population selection, not for prompt synthesis.

PDO: It optimizes a single prompt using preference-based feedback rather than explicit performance scores. It relies on pairwise win–loss judgments produced by an LLM acting as a judge (They don’t use ground-truth labels for evaluation of a prompt). Prompt optimization is framed as a dueling bandit problem and solved using Double Thompson Sampling. New prompt variants are generated using fixed, human-designed mutation templates applied to top-ranked prompts.

MIPRO MIPRO jointly optimizes multiple prompts across modules in a multi-stage LLM program. It generates candidate instructions and demonstrations independently from task descriptions or templates. No parent prompts or performance feedback are used during prompt generation. A Bayesian surrogate model searches over combinations of candidates using only a task-level evaluation metric.

GEPA GEPA is a multi-agent evolutionary framework designed for compound AI systems with multiple LLM modules. It treats the entire set of system prompts as a single evolving population rather than optimizing prompts independently. In each iteration, a candidate system is selected from the population using Pareto-based selection. Within the selected system, a target module and its corresponding prompt are chosen in a round-robin manner. Prompt mutations are guided by natural-language reflection over execution and evaluation traces from full system runs. This design promotes diversity across modules and helps avoid premature convergence to local optima.

References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2025.
- [2] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers, 2025.
- [3] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [4] Yuanchen Wu, Saurabh Verma, Justin Lee, Fangzhou Xiong, Poppy Zhang, Amel Awadelkarim, Xu Chen, Yubai Yuan, and Shawndra Hill. Llm prompt duel optimizer: Efficient label-free prompt optimization, 2025.
- [5] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers, 2024.